

采蘑菇平台软硬件控制方案计划

1. 项目目标

本项目计划搭建一套基于滑轨 SCARA 机械臂的采蘑菇平台，完成蘑菇目标识别、机械臂定位、吸盘采摘和搬运放置等基本功能。

系统采用以下执行机构：

关节	机械形式	执行器	控制方式
滑轨轴	水平直线运动	步进电机	STM32 或运动控制板
肩关节	SCARA 第一旋转关节	瓴控 MG4010E-i36	CAN 总线
肘关节	SCARA 第二旋转关节	瓴控 MG4010E-i36	CAN 总线
Z 轴	垂直直线运动	步进电机	STM32 或运动控制板
末端旋转轴	吸盘和 RealSense 旋转	总线舵机	USB 转总线控制板

整个机构可抽象为五个关节：

$$q = [s, \theta_1, \theta_2, z, \theta_w]$$

其中：

- s ：滑轨位置；
- θ_1 ：肩关节角度；
- θ_2 ：肘关节角度；
- z ：Z 轴高度；
- θ_w ：末端旋转角度。

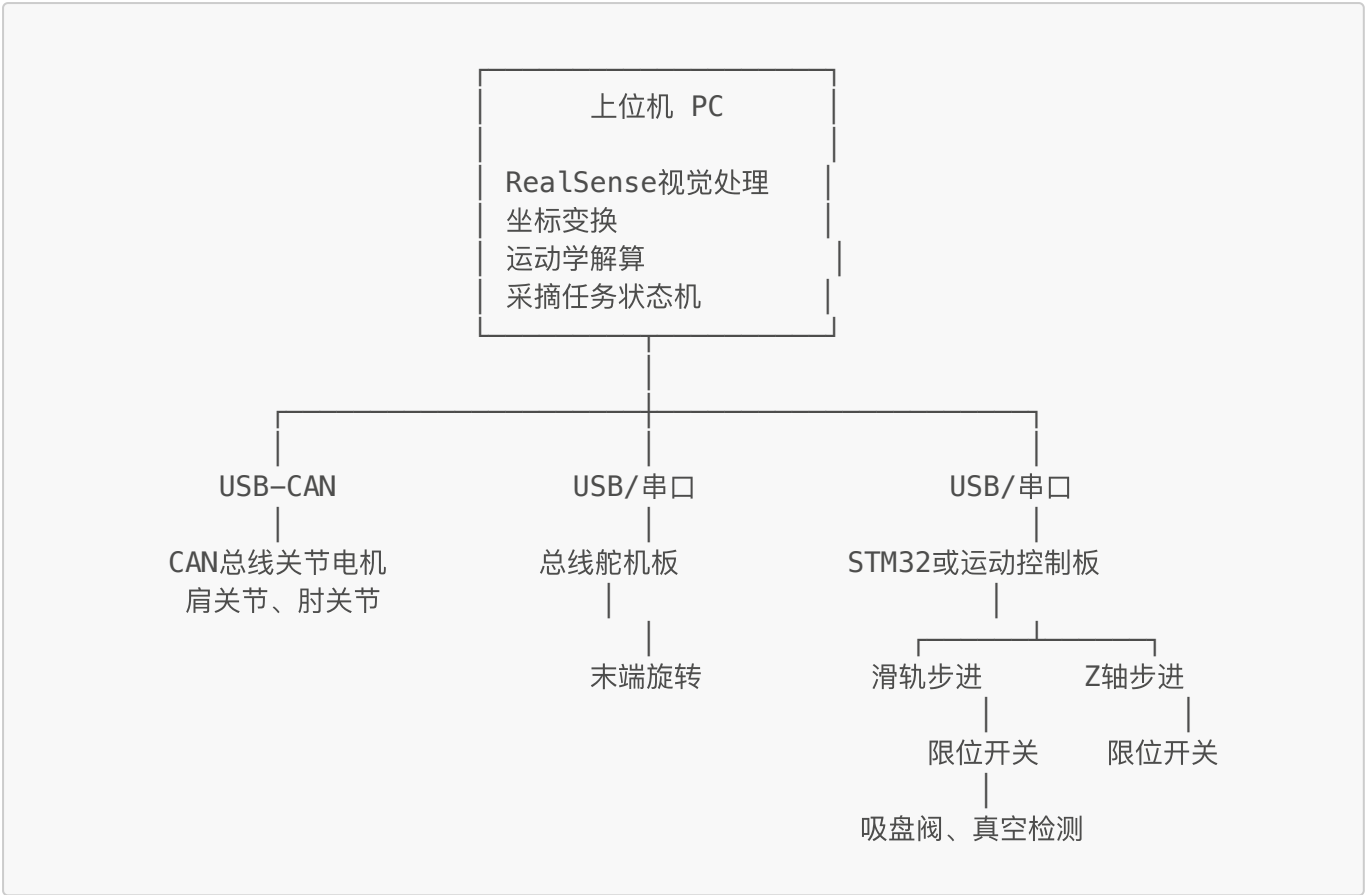
2. 总体实施思路

第一阶段以功能实现和快速调试为主要目标，不构建过于复杂的工业控制架构。

计划采用轻量化的混合控制方案：

- 上位机负责视觉处理、坐标变换、运动学解算、任务流程和总线设备控制；
- STM32 或专用运动控制板负责两个步进电机、限位开关、吸盘和安全输入输出；
- CAN 关节电机和末端总线舵机暂时由上位机直接控制；
- 后期如有更高实时性或产品化需求，再逐步将部分控制功能迁移到统一下位机。

总体结构如下：



3. 控制方案选择

3.1 上位机直接控制方案

上位机通过以下接口直接控制各执行器：

- USB-CAN 控制两个 MG4010E-i36 关节电机；
- USB 转总线控制板控制末端舵机；
- USB、串口或网络接口控制步进运动控制板；
- USB 3.x 接口连接 RealSense。

该方案结构简单、开发速度快，适合目前整体运行速度较低的采摘平台。

需要注意，步进电机不宜由 Python 程序通过 USB IO 逐个发送脉冲。USB 调度、操作系统线程调度和串口缓存会导致脉冲间隔不稳定，进而造成振动、丢步或位置误差。

因此，步进控制应采用以下方式之一：

1. 使用能够独立生成 STEP/DIR 脉冲和加减速曲线的运动控制板；
2. 使用 STM32 接收目标位置和速度，由 STM32 在本地生成步进脉冲。

上位机应发送类似以下命令：

```
MOVE RAIL 350.0 50.0 100.0
MOVE Z 80.0 20.0 50.0
```

而不是通过 USB 逐个产生 STEP 脉冲。

3.2 统一 STM32 下位机方案

完整的统一下位机方案可由 STM32 同时管理：

- 两个 CAN 关节电机；
- 两个步进电机；
- 总线舵机；
- 吸盘；
- 限位开关；
- 真空检测；
- 急停；
- 轨迹执行。

该方案实时性和系统一致性更好，但开发内容明显增加，包括多种通信协议、状态管理、轨迹控制和上位机通信协议设计。

对于当前低速原型平台，完整统一下位机带来的收益有限，因此暂不作为第一阶段方案。

3.3 推荐方案

第一版系统计划采用轻量 STM32 下位机方案：

上位机负责

- RealSense 数据采集；
- 蘑菇目标识别；
- 相机坐标到机器人坐标转换；
- 滑轨位置选择；
- SCARA 正逆运动学；
- CAN 关节控制；
- 总线舵机控制；
- 采摘任务状态机；
- 状态显示、日志和故障处理。

STM32 或运动控制板负责

- 滑轨步进脉冲；
- Z 轴步进脉冲；
- 加减速控制；
- 原点回零；
- 限位开关；
- 吸盘电磁阀；
- 真空压力检测；
- 看门狗；
- 急停和故障停止。

该方案能够控制开发工作量，同时保证步进控制和安全输入输出的可靠性。

4. 硬件连接计划

4.1 电源系统

计划将动力设备和控制设备分开供电：



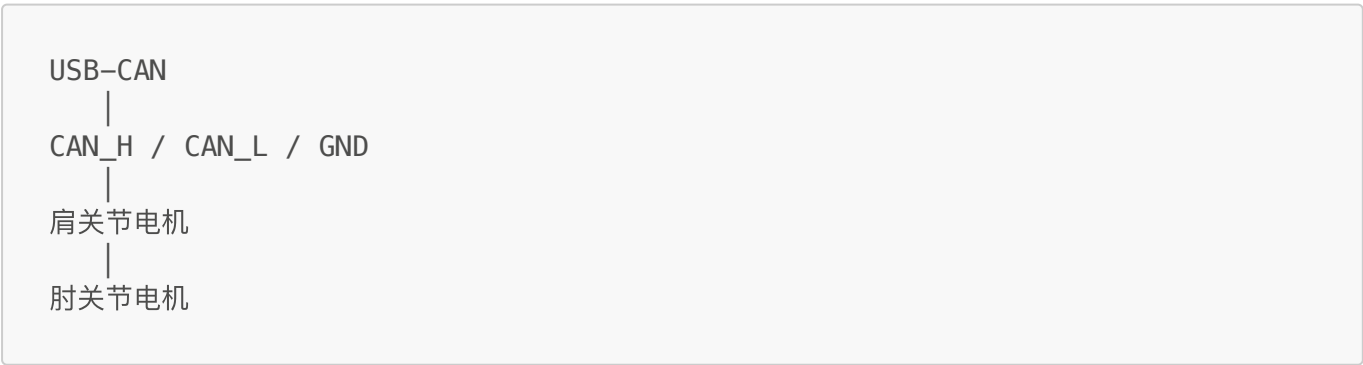
电源设计中需要重点考虑：

- 两个 CAN 关节电机的峰值电流；
- 步进电机启动和堵转电流；
- 总线舵机启动电流；
- 电磁阀和气泵等感性负载；
- 控制电源与动力电源之间的干扰；
- 急停时的动力切断方式。

RealSense 计划直接连接上位机 USB 3.x 接口，避免使用质量不稳定的 USB Hub。

4.2 CAN 总线连接

两个 MG4010E-i36 关节电机通过同一条 CAN 总线连接：



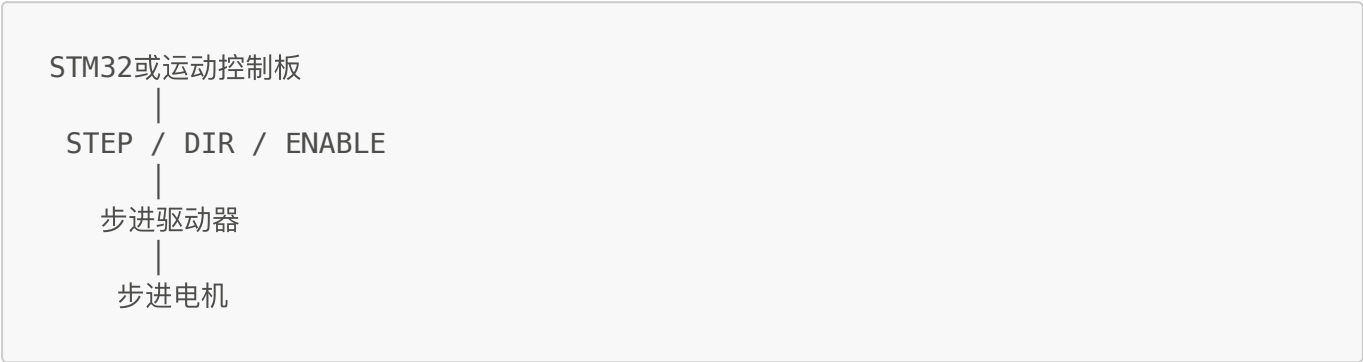
实施时需要：

- 为两个电机设置不同 CAN ID；
- 采用总线式串联连接，避免星形分叉；
- 在 CAN 总线两端配置 120 Ω终端电阻；

- 使用双绞线连接 CAN_H 和 CAN_L；
- 尽量将 CAN 信号线与电机动力线分开布置；
- 确认 USB-CAN 和电机之间的参考地连接方式；
- 优先使用带隔离功能的 USB-CAN。

4.3 步进电机连接

两个步进轴采用 STEP/DIR/ENABLE 控制：



滑轨和 Z 轴均计划配置：

- 原点限位开关；
- 末端限位开关；
- 软件最小位置；
- 软件最大位置；
- 驱动器使能控制；
- 急停输入；
- 回零流程。

由于普通步进电机没有位置反馈，系统上电后必须执行回零。发生碰撞、堵转或疑似丢步后，也需要重新回零。

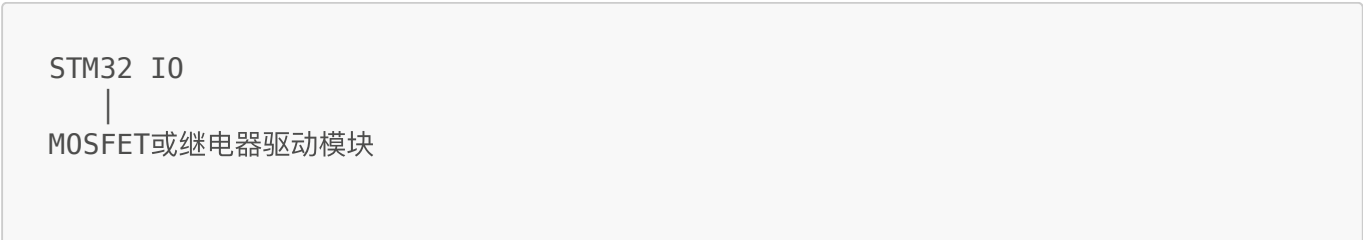
对于垂直 Z 轴，还需要评估断电下坠风险，可考虑：

- 带抱闸电机；
- 自锁丝杠；
- 配重；
- 气弹簧；
- 机械制动结构。

4.4 吸盘和真空系统

吸盘系统计划由 STM32 控制电磁阀或真空泵。

连接方式如下：



电磁阀或真空泵

STM32 IO 不应直接驱动电磁阀或气泵。

系统计划增加真空压力开关或真空传感器，用于判断蘑菇是否成功吸附，避免只根据吸盘开启状态推测采摘是否成功。

4.5 急停和安全回路

急停功能不依赖上位机软件单独实现。

急停触发后，应优先切断：

- CAN 关节电机动力或使能；
- 步进驱动器动力或使能；
- 吸盘执行输出。

上位机可以保持供电，用于显示故障状态、保存日志和执行故障恢复。

5. 软件总体分层

软件计划采用五层结构，但各层保持简单，不引入过度工程化设计。

第5层：采摘任务层 检测蘑菇、接近、下降、吸取、搬运、放置
第4层：运动控制层 move_pose、move_joint、home、stop
第3层：运动学和坐标变换层 滑轨选择、SCARA逆解、相机坐标转换
第2层：关节抽象层 rail、shoulder、elbow、z_axis、wrist
第1层：硬件驱动层 USB-CAN、串口、STM32协议、RealSense

6. 硬件驱动层计划

硬件驱动层只负责具体设备通信，不处理运动学和采摘任务。

计划实现以下驱动模块：

```
CanMotorDriver
StepperControllerDriver
BusServoDriver
VacuumIODriver
RealSenseDriver
```

各驱动模块主要负责：

- 打开和关闭设备；
- 发送底层通信数据；
- 接收设备反馈；
- 数据帧校验；
- 超时检测；
- 重连处理；
- 编码器、脉冲和协议单位转换；
- 故障码读取。

驱动层不直接处理蘑菇坐标，也不决定关节目标位置。

7. 关节抽象层计划

所有执行器计划统一抽象为关节，使上层不需要关心 CAN 数据帧、步进脉冲或舵机寄存器。

每个关节提供统一接口：

```
joint.enable()
joint.disable()
joint.home()
joint.move_to(position, velocity, acceleration)
joint.get_position()
joint.is_moving()
joint.stop()
joint.get_fault()
```

关节层统一处理：

- 单位转换；
- 转动方向；
- 零位偏置；
- 位置限位；
- 速度限制；
- 加速度限制；
- 通信故障；
- 状态缓存。

每个关节计划配置以下通用参数：

```
name
joint_type
direction
zero_offset
min_position
max_position
max_velocity
max_acceleration
```

步进关节额外配置：

```
steps_per_revolution
microstep
screw_pitch
steps_per_mm
home_direction
home_speed
```

CAN 关节额外配置：

```
can_id
can_bitrate
gear_ratio
motor_zero_offset
position_command_range
```

上层统一使用：

- 直线关节：mm 或 m；
- 旋转关节：rad。

8. 运动学计划

8.1 正运动学

设滑轨沿机器人 X 方向运动，两个 SCARA 连杆长度分别为 L_1 和 L_2 。

末端平面位置为：

$$x = s + L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2)$$

$$y = L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2)$$

末端方向为：

$$\phi = \theta_1 + \theta_2 + \theta_w$$

Z 轴位置根据实际机械安装方向转换为末端高度。

8.2 SCARA 逆运动学

给定滑轨位置 s 后，首先计算目标点相对于 SCARA 底座的位置：

$$x_r = x - s$$

然后计算：

$$c_2 = \frac{x_r^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}$$

$$\theta_2 = \text{atan2} \left(\pm \sqrt{1 - c_2^2}, c_2 \right)$$

$$\theta_1 = \text{atan2}(y, x_r) - \text{atan2}(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2)$$

末端旋转轴角度为：

$$\theta_w = \phi - \theta_1 - \theta_2$$

逆运动学通常存在左肘和右肘两组解。软件计划根据以下条件选择：

- 是否超过关节限位；
- 是否接近奇异位置；
- 是否存在机械碰撞风险；
- 是否与当前姿态接近；
- 是否需要过大的滑轨移动。

9. 滑轨冗余处理计划

滑轨和两个 SCARA 关节都能够改变末端 X 方向位置，因此同一个末端目标可能对应多组关节解。

第一版不使用复杂优化器，采用一维候选搜索：

1. 在滑轨允许范围内生成多个候选位置；
2. 对每个滑轨候选位置计算 SCARA 逆解；
3. 删除无解或超过限位的结果；
4. 计算每组解的代价；
5. 选择代价最低的目标关节组合。

代价函数可采用：

$$J = w_s |s - s_{\text{current}}| + w_q \|q - q_{\text{current}}\| + w_l P_{\text{limit}} + \frac{w_g}{|\sin \theta_2| + \epsilon}$$

该方法主要用于：

- 减少滑轨移动距离；
- 减少关节转动；
- 避免接近关节限位；
- 避免手臂完全伸直或完全折叠；

- 提高运动连续性。

由于平台运行速度较慢，一维搜索的计算量可以接受。

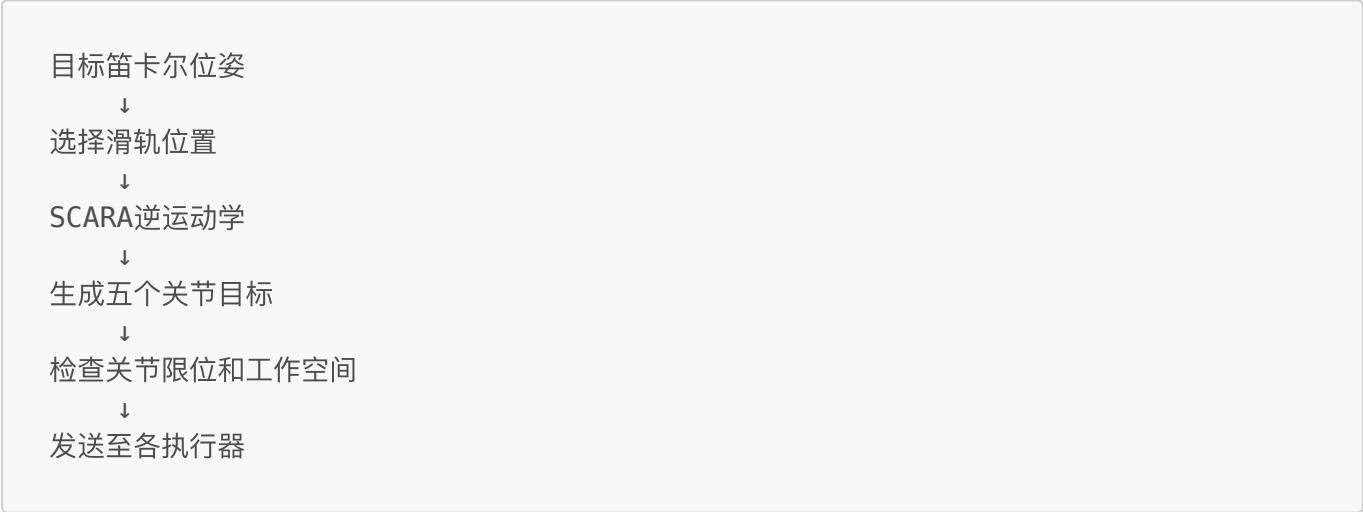
10. 运动控制计划

第一阶段采用点到点运动，不立即实现复杂的五轴同步轨迹规划。

计划提供以下上层接口：

```
robot.home()  
robot.move_joint(joint_targets)  
robot.move_pose(x, y, z, yaw)  
robot.set_vacuum(True)  
robot.stop()
```

`move_pose()` 内部执行流程如下：



第一版采摘动作采用分阶段执行：

1. Z 轴升到安全高度；
2. 移动滑轨；
3. 移动肩关节和肘关节；
4. 调整末端旋转轴；
5. Z 轴下降；
6. 开启吸盘；
7. 检查真空状态；
8. Z 轴上升；
9. 移动至收集位置；
10. 关闭吸盘。

该方式不依赖严格的实时同步，便于调试和故障定位。

11. RealSense 和吸盘坐标计划

RealSense 和吸盘固定在末端旋转平台上，属于眼在手上结构。

计划定义以下坐标系：

```
world
rail_base
scara_base
link1
link2
wrist
camera
tcp
```

其中：

- **camera** 为 RealSense 坐标系；
- **tcp** 为吸盘工具中心点坐标系；
- **wrist** → **camera** 为固定变换；
- **wrist** → **tcp** 为固定变换。

RealSense 识别得到蘑菇在相机坐标系中的位置 p_C ，通过当前关节状态和相机外参转换为机器人基座坐标：

$$p_B = T_B^C(q)p_C$$

然后将机器人坐标系中的目标位置输入逆运动学模块。

系统需要完成以下标定：

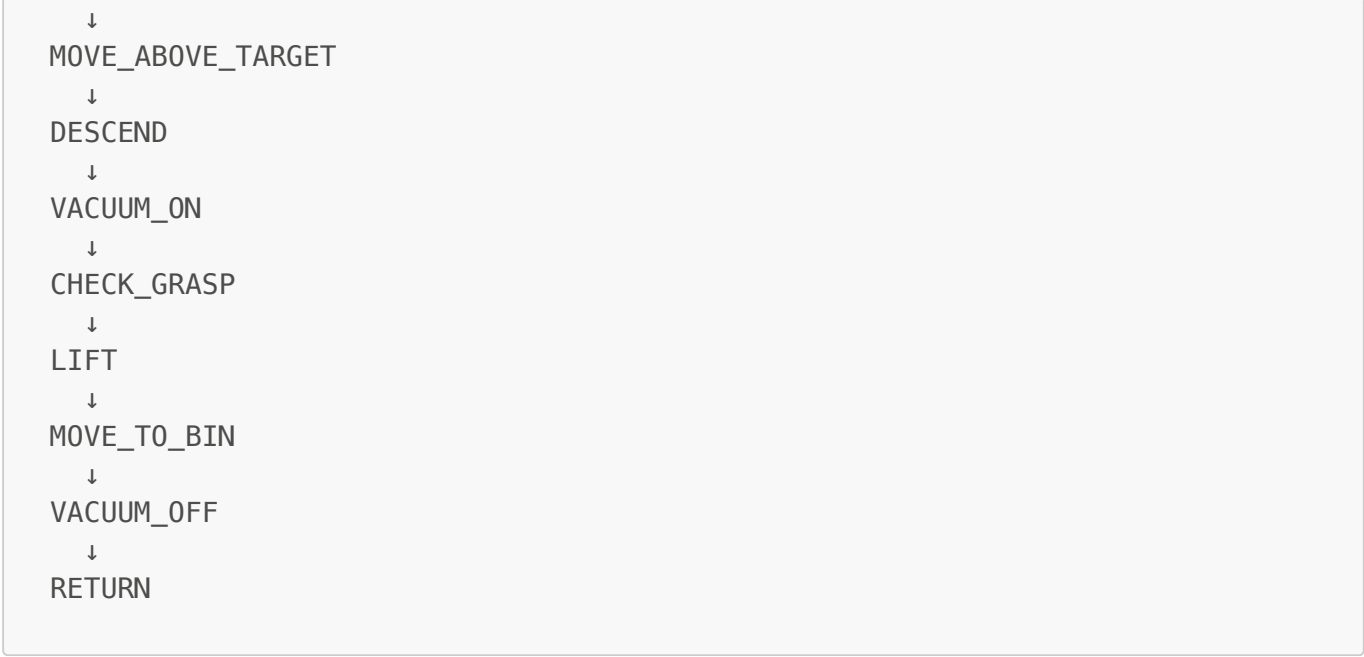
- 两个 SCARA 连杆长度；
- 滑轨零点；
- 肩关节和肘关节机械零位；
- Z 轴零点；
- 末端舵机零位；
- RealSense 相对于 wrist 的位姿；
- 吸盘 TCP 相对于 wrist 的位姿。

如果末端没有安装滑环，需要限制舵机旋转范围，避免 RealSense 的 USB 线反复扭转。

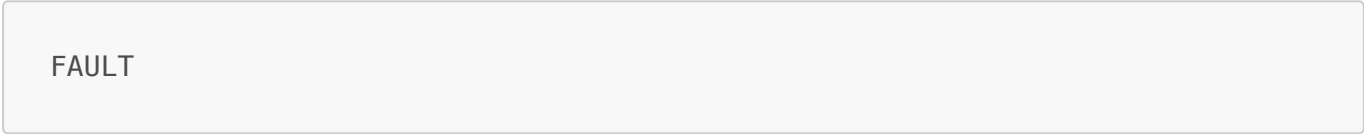
12. 采摘任务状态机计划

采摘流程计划使用简单有限状态机实现：

```
IDLE
↓
HOMING
↓
WAIT_TARGET
↓
COMPUTE_TARGET
```



发生异常时统一进入：



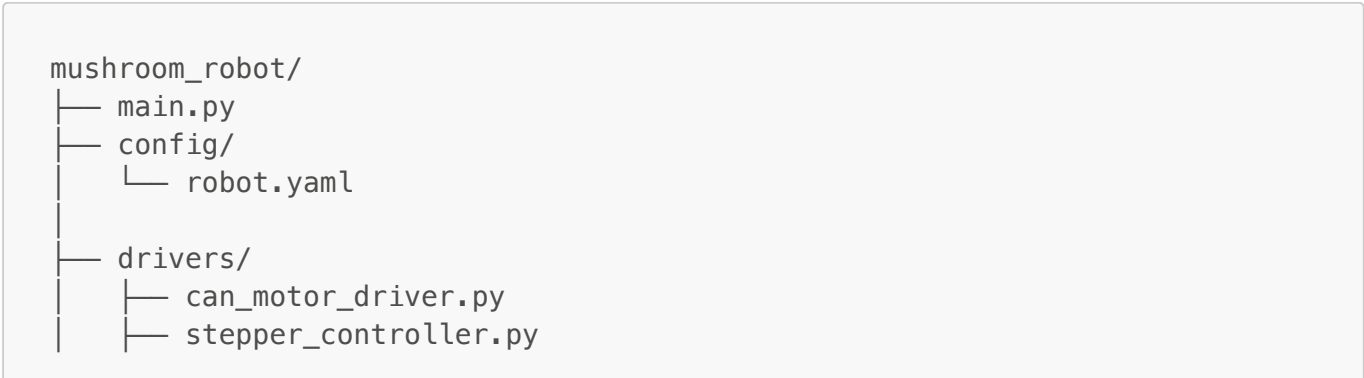
计划监控以下故障条件：

- CAN 电机失联；
- CAN 电机返回故障码；
- 步进轴异常触发限位；
- 运动超时；
- 逆运动学无解；
- 目标超出工作空间；
- 真空建立失败；
- 急停触发；
- STM32 看门狗超时。

任务层只决定下一步动作，不直接操作 CAN 数据帧或步进脉冲。

13. 软件目录计划

第一版软件保持单项目、少进程和低耦合结构：



```
├── bus_servo_driver.py
├── vacuum_io.py
├── realsense_camera.py
├── robot/
│   ├── joint.py
│   ├── robot_state.py
│   └── mushroom_robot.py
├── kinematics/
│   ├── scara_kinematics.py
│   ├── rail_selector.py
│   └── transforms.py
├── motion/
│   └── motion_controller.py
└── tasks/
    └── pick_mushroom.py
```

第一阶段不计划引入：

- 复杂插件机制；
- 多层消息中间件；
- 每个关节独立进程；
- 完整工业机器人控制框架；
- 复杂轨迹优化；
- 过多抽象层。

建议运行结构为：

- 一个主控制进程；
- 一个 CAN 通信线程；
- 一个 RealSense 数据线程；
- 一个 STM32 或运动板通信线程；
- 一个采摘任务状态机。

14. 开发阶段计划

阶段一：单设备测试

1. 测试两个 MG4010E-i36；
2. 完成电机使能；
3. 完成位置控制；
4. 读取位置、速度和故障状态；
5. 测试紧急停止；
6. 测试末端总线舵机；
7. 测试吸盘电磁阀和真空传感器。

阶段二：步进轴控制

1. 完成滑轨 STEP/DIR 控制；
2. 完成 Z 轴 STEP/DIR 控制；
3. 实现加减速；
4. 实现原点回零；
5. 实现软限位和硬限位；
6. 测试断电和急停行为；
7. 测试 Z 轴防坠落能力。

阶段三：关节抽象

1. 为五个关节建立统一接口；
2. 配置方向、零位和限位；
3. 统一位置和速度单位；
4. 实现状态读取和故障处理；
5. 实现整机回零流程。

阶段四：运动学验证

1. 测量两个连杆长度；
2. 建立 SCARA 正运动学；
3. 建立 SCARA 逆运动学；
4. 验证左肘和右肘两组解；
5. 加入关节限位检查；
6. 加入滑轨候选搜索；
7. 通过人工输入坐标测试末端到达能力。

阶段五：运动控制

1. 实现 `move_joint()`；
2. 实现 `move_pose()`；
3. 实现分阶段点到点运动；
4. 增加运动超时；
5. 增加停止和急停；
6. 增加简单日志记录。

阶段六：视觉集成

1. 接入 RealSense；
2. 定义 camera 和 tcp 坐标系；
3. 标定相机相对于 wrist 的位姿；
4. 标定吸盘 TCP；
5. 将视觉识别结果转换到机器人坐标系；
6. 使用固定蘑菇目标测试自动定位。

阶段七：完整采摘流程

1. 实现采摘状态机；

2. 实现目标上方安全点；
3. 实现下降和吸附；
4. 实现真空检测；
5. 实现搬运和放置；
6. 实现失败重试和故障退出；
7. 测试连续采摘。

15. 第一版系统边界

第一版系统计划实现：

- 五个关节的统一控制；
- 滑轨和 Z 轴自动回零；
- SCARA 正逆运动学；
- 滑轨冗余位置选择；
- RealSense 坐标转换；
- 吸盘控制和真空检测；
- 基本采摘状态机；
- 限位、急停、超时和通信故障处理。

第一版暂不实现：

- 五轴严格同步插补；
- 高速连续轨迹规划；
- 复杂碰撞检测；
- 动力学控制；
- CANopen 完整协议栈；
- ROS 2 Control 和 MoveIt 集成；
- 复杂任务调度；
- 完整工业控制器架构。

后续可以根据运行效果，再决定是否增加同步轨迹、视觉伺服、碰撞检测或统一 STM32 下位机。

16. 方案结论

本项目第一阶段计划采用功能优先的轻量控制结构：

上位机负责 RealSense 视觉、坐标变换、滑轨位置选择、SCARA 逆运动学、CAN 关节控制、末端舵机控制和采摘任务流程；STM32 或运动控制板负责两个步进轴、限位开关、吸盘、真空检测、急停和看门狗。

所有执行器在软件中统一抽象为关节。上层只使用位置、速度、加速度和状态接口，不直接处理 CAN 数据帧、步进脉冲或舵机寄存器。

该方案能够以较小的开发工作量实现完整采摘功能，同时保留后续向统一下位机、ROS 2 或更复杂轨迹控制扩展的空间。

17. 术语对照表

中文术语	English Term	Abbreviation
选择顺应性装配机械臂	Selective Compliance Assembly Robot Arm	SCARA
控制器局域网	Controller Area Network	CAN
通用串行总线	Universal Serial Bus	USB
实感深度相机	Intel RealSense Depth Camera	RealSense
关节抽象	Joint Abstraction	无常用缩写
正运动学	Forward Kinematics	FK
逆运动学	Inverse Kinematics	IK
工具中心点	Tool Center Point	TCP
点到点运动	Point-to-Point Motion	PTP
有限状态机	Finite-State Machine	FSM
脉冲和方向控制	Step and Direction Control	STEP/DIR
输入输出	Input/Output	IO
看门狗	Watchdog Timer	WDT
眼在手上	Eye-in-Hand	无常用缩写
坐标变换	Coordinate Transformation	无常用缩写
终端电阻	Termination Resistor	无常用缩写
运动控制器	Motion Controller	无常用缩写